From API Calls to Agents — First Principles

Complete Beginner-Friendly Notes

Topics covered: LLM APIs, Temperature, Multi-turn Chat, Function Calling, ReAct Agents, Code Agents, Native Tool Calling

---

The Big Picture — What Are We Building?

In the last class, you built a mini LLM from scratch and saw how it generates text. Now we flip to the other side: **using** an LLM through an API, and then building an **agent** — a program where the LLM can think, use tools, see results, and repeat until a task is done.

Think of it this way: the last class was like building a car engine. This class is about learning to drive — and eventually building a self-driving car.

By the end, you'll have built (in ~60 lines of Python) the same pattern that powers Claude Code, OpenAI Codex, and every other AI agent.

The Evolution: Autocomplete → Chat → Agent

AI coding tools have evolved through three distinct eras:

2021 — Autocomplete (GitHub Copilot): AI suggests the next line of code inside your editor. You're still driving — the AI is just a smart autocomplete. Like a GPS that only shows the next turn.

2023 — Chat Assistant (ChatGPT/Claude): You describe a problem in conversation, the AI gives you code blocks, and you manually copy-paste them into your project. Like asking a knowledgeable friend for help, then doing the work yourself.

2025 — Agent (Claude Code/Codex): You give the AI a goal, and it plans, writes code, runs tests, sees errors, fixes them, and keeps going until the job is done. Like hiring a developer who works autonomously.

The key shift at each step: **the human does less manual work, the AI takes on more autonomy.**

Part 1: Your First API Call

What is an API Call?

When you use ChatGPT or Claude through their website, there's an API (Application Programming Interface) behind the scenes. An API is just a way to send a request to a server and get a response back — like ordering food through a delivery app instead of going to the restaurant.

The notebook uses **OpenRouter** — a service that gives you access to many different LLMs through one API key, including free models. It's "OpenAI-compatible," meaning the code looks identical to OpenAI's SDK, just with a different URL.

Setup

```
from openai import OpenAI

client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=os.environ["OPENROUTER_API_KEY"],
)
FREE_MODEL = "openrouter/free"
```

This creates a "client" — your connection to the LLM. Think of it like opening a phone line to the AI.

Making a Call

```
response = client.chat.completions.create(
    model=FREE_MODEL,
    messages=[{"role": "user", "content": "What is 2 + 2? Ans
wer in one word."}],
    temperature=0.0,
```

```
    max_tokens=50,
)
print(response.choices[0].message.content)  # "Four"
```

What's happening under the hood: You send tokens to the server, the LLM processes them (exactly like the model you built in Class 5), and sends tokens back. The API just wraps this process in an HTTP request.

The response also tells you how many tokens were used — this matters because API usage is billed per token.

Temperature — You Already Know This!

Remember from Class 5: `softmax(logits / temperature)`. The same concept applies here through the API.

- **T=0:** Deterministic — same input always gives the same output. The model always picks the highest-probability token.
- **T=1:** Natural variety — the model samples normally from the probability distribution.
- **T=2:** Chaos — even low-probability tokens get picked, leading to creative but often nonsensical output.

The notebook demonstrates this by generating the same prompt 3 times at each temperature. At T=0, all 3 outputs are identical. At T=2, they're wildly different.

The Model Has NO Memory

This is a critical insight that most beginners miss: **the LLM remembers nothing between API calls.** Every time you send a message, you must resend the ENTIRE conversation history.

```
# Turn 1: Send initial question
conversation = [
    {"role": "system", "content": "You are a math tutor."},
    {"role": "user", "content": "What is a derivative?"},
]
r1 = client.chat.completions.create(model=FREE_MODEL, message
```

```
s=conversation)

# Turn 2: Append the assistant's reply AND your new question
conversation.append({"role": "assistant", "content": r1.choic
es[0].message.content})
conversation.append({"role": "user", "content": "Give me an e
xample with x².")})
r2 = client.chat.completions.create(model=FREE_MODEL, message
s=conversation)
```

Notice the prompt tokens GROW from Turn 1 to Turn 2. That's because Turn 2 re-sends everything from Turn 1 plus the new messages. What feels like "memory" in ChatGPT is actually the app re-sending the full conversation every time.

This is the **context engineering problem** — as conversations get longer, you're sending (and paying for) more and more tokens.

Part 2: Function Calling — The Bridge to Agents

The Key Insight

The LLM can't actually DO anything in the real world. It can't check the weather, access a database, or run code. It can only generate text.

Function calling is a clever trick: we describe available tools to the model, and when it needs one, it outputs a structured JSON request saying "I want to call this function with these arguments." Then **YOUR code** actually executes the function and feeds the result back.

The model never "calls" anything. It generates text that describes what it wants. Your code does the actual work.

Defining Tools

```
tools = [
    {
        "type": "function",
        "function": {
```

```
            "name": "get_weather",
            "description": "Get current weather for a city.",
            "parameters": {
                "type": "object",
                "properties": {
                    "city": {"type": "string", "description":
"City name"}
                },
                "required": ["city"]
            }
        }
    }
]
```

This is a "menu" we hand to the LLM. It tells the model: "Here's a tool called `get_weather`. It takes a city name as input." The model doesn't HAVE this tool — it just knows it can REQUEST it.

The Full Tool Call Cycle

When the user asks "What's the weather in Tokyo?", here's what happens:

1. **User sends question** → API call with the tools menu
2. **Model decides it needs a tool** → Instead of answering directly, it outputs: `{"name": "get_weather", "arguments": {"city": "Tokyo"}}`
3. **Your code runs the actual function** → Gets the real weather data
4. **Result sent back to model** → The model now has real data in its context
5. **Model generates final answer** → "The weather in Tokyo is 22°C and partly cloudy."

The `finish_reason` in the API response tells you what happened: `"stop"` means the model answered normally, `"tool_calls"` means it wants to use a tool.

```
# After model requests a tool:
tc = msg.tool_calls[0]
fn = TOOL_FNS[tc.function.name]                    # Look up
```

```
the real function
result = fn(**json.loads(tc.function.arguments))     # Execut
e it
# Feed result back to the model as a "tool" message
messages.append({"role": "tool", "tool_call_id": tc.id, "cont
ent": result})
```

This cycle — User → Model requests tool → Code runs tool → Result back to model → Final answer — is the foundation of everything that follows.

Part 3: Building a ReAct Agent From Scratch

What is an Agent?

An agent is simply: **LLM + Tools + Loop**

- **LLM:** The brain that reasons and makes decisions
- **Tools:** The hands that interact with the world (search Wikipedia, do math, run code)
- **Loop:** The persistence that keeps going until the task is done

That's it. No magic. No special AI technology. Just a program that calls an LLM in a loop, letting it use tools until it has enough information to answer.

The ReAct Pattern

ReAct stands for **Reasoning + Acting** (from a 2022 research paper). The agent alternates between thinking and doing:

1. **THOUGHT:** "I need to find out when Shakespeare was born."
2. **ACTION:** `search_wikipedia("William Shakespeare")`
3. **OBSERVATION:** `{"title": "William Shakespeare", "summary": "Born 1564..."}`
4. **THOUGHT:** "Now I need Leonardo da Vinci's birth year."
5. **ACTION:** `search_wikipedia("Leonardo da Vinci")`
6. **OBSERVATION:** `{"title": "Leonardo da Vinci", "summary": "Born 1452..."}`

7. **THOUGHT:** "Shakespeare born 1564, da Vinci born 1452. Da Vinci was first, by 112 years."
8. **FINAL_ANSWER:** "Leonardo da Vinci was born first, 112 years before Shakespeare."

Each step is just text. The "THOUGHT" is the model generating reasoning tokens. The "ACTION" is the model outputting a tool request in a specific format. The "OBSERVATION" is your code running the tool and pasting the result back.

Why Loops Beat One-Shot

Why can't we just ask the model to answer in one go? Three reasons:

Real-world feedback: The model's training data is static and outdated. Tools give it access to live information it doesn't have in its weights.

Iterative correction: If the first search doesn't find the right page, the agent can try a different search term. Like a human developer who runs code, sees an error, and fixes it.

Persistence: Complex tasks (like "compare two historical figures") require multiple steps that can't be compressed into one prompt. The loop lets the agent work through them sequentially.

The System Prompt IS the Architecture

```
REACT_SYSTEM_PROMPT = """You are a helpful assistant that sol
ves problems step by step.

You have access to these tools:
{tool_descriptions}

When you need to use a tool:
THOUGHT: <your reasoning>
ACTION: <tool_name>
ACTION_INPUT: <arguments as JSON>

When you have the final answer:
THOUGHT: <your reasoning>
```

```
FINAL_ANSWER: <your complete answer>
"""
```

This prompt is the entire "architecture" of our agent. It tells the LLM to follow a specific format — THOUGHT, then ACTION or FINAL_ANSWER. Our code can then parse this format with simple regex to figure out what the model wants to do.

The Real Tools

The notebook uses three real tools:

- `search_wikipedia(query)` — Hits the actual Wikipedia API and returns a summary. This is a real HTTP request to a real server.
- `calculate(expression)` — Evaluates math expressions safely.
- `get_current_date()` — Returns today's date and time.

The Agent Loop (~60 Lines)

```
def run_agent(user_query, max_iterations=10):
    # Build system prompt with tool descriptions
    # Start message list with system + user query

    for i in range(max_iterations):
        # 1. Ask the LLM what to do next
        response = client.chat.completions.create(...)
        text = response.choices[0].message.content

        # 2. Parse: does it contain FINAL_ANSWER?
        if "FINAL_ANSWER:" in text:
            return the answer  # We're done!

        # 3. Parse: does it contain ACTION?
        #    Extract tool name and arguments using regex

        # 4. Execute the tool
        observation = TOOLS[tool_name]["fn"](**args)
```

```
        # 5. Feed observation back as a user message
        messages.append({"role": "user", "content": f"OBSERVA
TION: {observation}"})

        # Loop continues — model now sees the observation
```

That's the whole thing. The same pattern — think, act, observe, repeat — powers Claude Code and every other AI agent. The difference is scale and polish, not architecture.

What Happens Step by Step

For the query "What is 123 × 123?":

```
--- Iteration 1 ---
💭 THOUGHT: I need to calculate 123 times 123.
🔧 ACTION: calculate({"expression": "123*123"})
👁️ OBSERVATION: {"expression": "123*123", "result": 15129}

--- Iteration 2 ---
💭 THOUGHT: I have the result.
✅ FINAL_ANSWER: 123 × 123 = 15,129
```

For a harder multi-hop question like "Who was born first, Shakespeare or Da Vinci?", the agent chains multiple Wikipedia searches and a calculation across 4-5 iterations.

Part 4: Tracing the Agent

The Context Engineering Problem

Every iteration of the agent loop re-sends the ENTIRE conversation history — system prompt, user query, every THOUGHT, every ACTION, every OBSERVATION from every previous iteration.

The traced version of the agent logs token counts per iteration:

```
Iter 1:   450 in +  120 out =   570
Iter 2:   690 in +  100 out =   790
Iter 3:   990 in +   80 out =  1070
Iter 4:  1200 in +  150 out =  1350

TOTAL: 3,780 tokens
```

See how `prompt_tokens` grows each iteration? That's because we're stuffing more and more history into the context window. For a 10-iteration agent run, the last iteration might send 5x more tokens than the first.

This is why production agents need **context management** — summarizing old history, dropping irrelevant observations, or compressing the conversation. Without it, costs and latency balloon.

Part 5: Code Agent — Claude Code in 60 Lines

Giving the Agent Developer Tools

Now we upgrade the agent with tools that a software developer would use:

- `read_file(path)` — Read a file's contents
- `write_file(path, content)` — Create or modify a file
- `run_python(code)` — Execute Python code and get stdout/stderr
- `list_files(path)` — See what files exist in a directory

These are essentially the same tools that Claude Code has. With just these four capabilities, the agent can write code, save it to a file, run it, see if it works, and fix errors if it doesn't.

How It Solves a Coding Task

Task: "Write a function `is_prime(n)` and test it."

```
--- Iteration 1 ---
💭 THOUGHT: I need to write the is_prime function and save it.
🔧 ACTION: write_file({"path": "prime.py", "content": "def is_prime(n):..."})
👁️ OBSERVATION: {"status": "success", "path": "prime.py"}
```

```
--- Iteration 2 ---
💭 THOUGHT: Now I need to write a test script.
🔧 ACTION: write_file({"path": "test_prime.py", "content": "from prime import..."})
👁️ OBSERVATION: {"status": "success"}

--- Iteration 3 ---
💭 THOUGHT: Let me run the test.
🔧 ACTION: run_python({"code": "exec(open('test_prime.py').read())"})
👁️ OBSERVATION: {"stdout": "1: False\n2: True\n13: True\n15: False\n97: True\n100: Fals
e"}

--- Iteration 4 ---
💭 THOUGHT: All tests passed. The function works correctly.
✅ FINAL_ANSWER: Created prime.py with is_prime() function. All tests passed.
```

The critical instruction in the code agent's system prompt: **"After writing code, always run it to test. If a test fails, read the error, fix the code, try again."** This is what makes it an agent rather than just a code generator — it debugs its own work.

Code Agent System Prompt

```
CODE_SYSTEM_PROMPT = """You are a coding agent. You write, ru
n, and debug Python.

Available tools:
{tool_descriptions}

Rules:
- ONE action per turn. Wait for OBSERVATION.
- After writing code, always run_python to TEST it.
- If a test fails, read the error, fix the code, try again.
- Include print() statements so you can see output.
"""
```

The system prompt tells the model to behave like a careful developer: write code, test it, fix bugs, verify results. The loop gives it the ability to actually iterate on failures.

Part 6: Native Function Calling

Regex Parsing vs Native API

In Part 3, our agent outputs text like `ACTION: search_wikipedia` and we parse it with regex. This works but is fragile — the model might format things slightly differently and break our parser.

Modern APIs support **native function calling**: instead of the model outputting text that we parse, the API returns structured `tool_calls` objects directly. No regex needed.

```
# Regex-based (Part 3):
action_match = re.search(r"ACTION:\s*(\w+)", text)    # Fragi
le!

# Native (Part 6):
msg.tool_calls[0].function.name         # Clean, structured
msg.tool_calls[0].function.arguments    # Already JSON
```

Native Agent Loop

The native version is cleaner because the API handles the formatting:

```
def run_native_agent(user_query, max_iterations=10):
    for i in range(max_iterations):
        response = client.chat.completions.create(
            model=TOOL_MODEL, messages=messages,
            tools=NATIVE_TOOLS,   # Pass tool definitions
        )
        msg = response.choices[0].message
        finish = response.choices[0].finish_reason

        if finish == "stop" and msg.content:
            return msg.content   # Model answered — done!

        if msg.tool_calls:       # Model wants to use tools
```

```
            for tc in msg.tool_calls:
                result = NATIVE_FNS[tc.function.name](**json.
loads(tc.function.arguments))
                messages.append({"role": "tool", "tool_call_i
d": tc.id, "content": result})
```

The key difference: instead of the model generating `THOUGHT: ... ACTION: search_wikipedia ACTION_INPUT: {"query": "France"}` as text, the API returns a structured object with `tool_calls`. The model can also call **multiple tools in one turn** with native function calling.

This is closer to how production agents work — less string parsing, more structured communication.

Claude Code's Toolset

The slides mention that Claude Code — the real production agent — uses just 4 core tools:

Tool	What It Does
`read_file(path)`	Read a file to understand the codebase
`write_file(path, content)`	Create or modify code files
`run_command(cmd)`	Execute bash commands (tests, builds, git)
`search_code(query)`	Search through a large codebase with grep/ripgrep

Notice anything? These are almost identical to the tools we built in Part 5. Claude Code is fundamentally the same architecture — LLM + tools + loop — just with a more polished implementation, better prompting, and a lot of engineering around edge cases.

Master Summary

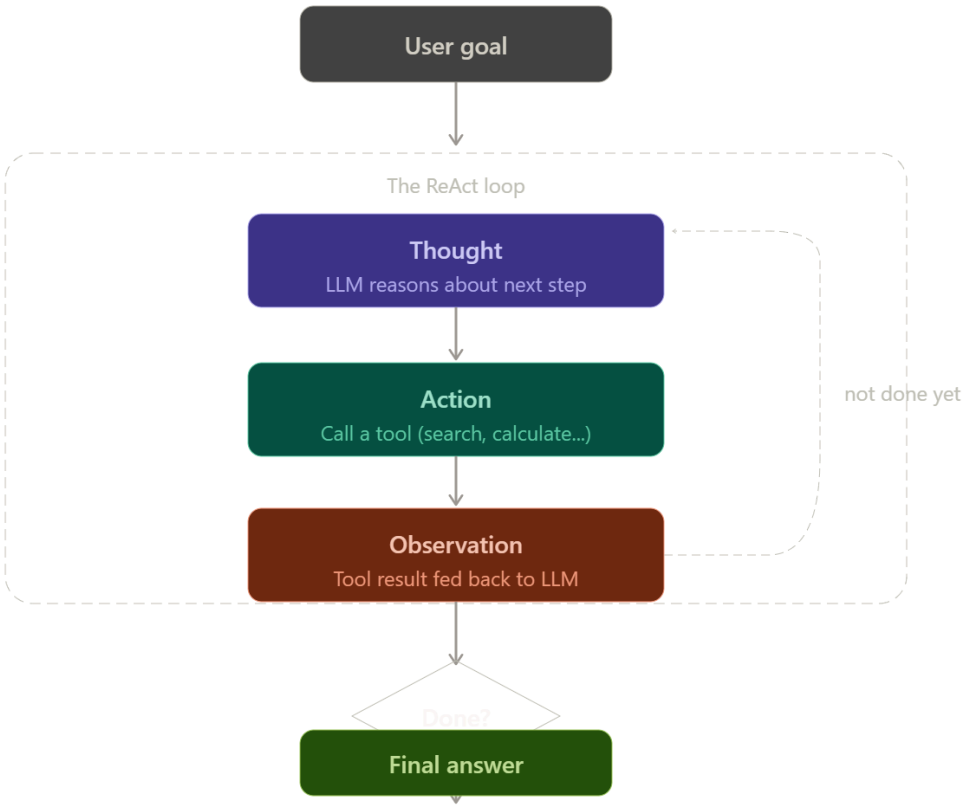
Concept	What It Is	Analogy
API Call	Send tokens to an LLM server, get tokens back	Ordering food through a delivery app

Concept	What It Is	Analogy
Temperature	Controls randomness of token sampling (same `softmax(logits/T)` from Class 5)	Thermostat for creativity
Multi-turn Chat	Re-send the full conversation every time (model has no memory)	Retelling the entire story each time you continue
Function Calling	Model outputs JSON requesting a tool; your code executes it	Writing a shopping list for someone else to buy
ReAct Agent	LLM + Tools + Loop following Think→Act→Observe→Repeat	A detective investigating a case step by step
Code Agent	ReAct agent with file I/O and code execution tools	A junior developer who writes, tests, and debugs autonomously
Native Tool Calling	API returns structured tool requests instead of text to parse	A phone menu with buttons vs. shouting commands
Context Engineering	Managing the growing token count as agent history accumulates	Keeping meeting notes concise so the agenda doesn't take all day

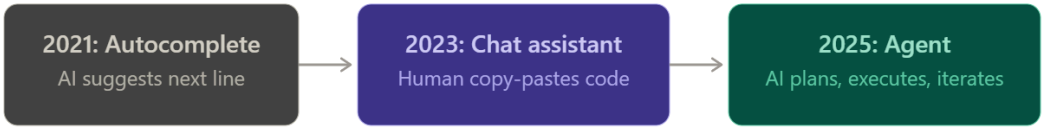
TODOs from the Lecture

1. **Context Management:** Add summarization — after N iterations, compress conversation history. Compare token usage before/after.
2. **Planning Step:** Make the agent first create a PLAN (list of steps), then execute each one. Compare Plan→Execute vs pure ReAct.
3. **Multi-Agent:** Build a "researcher" agent (Wikipedia tools) and a "coder" agent (file/code tools) with an orchestrator that routes tasks.
4. **Web Browsing:** Add a `fetch_webpage(url)` tool so the agent can follow links. Explore what new failure modes appear.

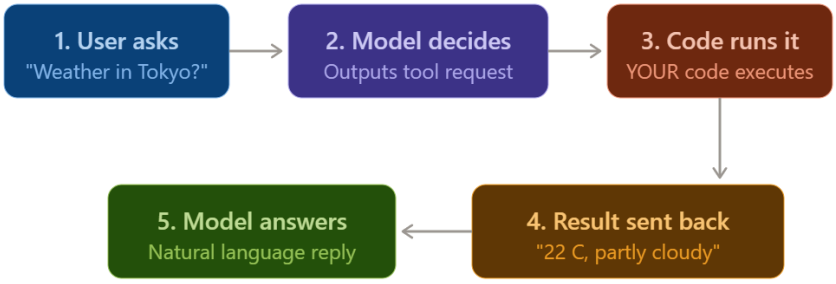
KEY DIAGRAMS:



Here's how the agent thinks, acts, and observes in a loop. Now the evolution of AI coding tools and the tool call cycle:



The function calling cycle



Agent = this cycle in a loop

Same pattern behind Claude Code, Codex, and every AI agent